# Odin OS: Cryptographically Authorized Memory Mutation in a RISC-V PMP-Enforced Agent Execution Environment

Nathan Walker
Ravenhelm Platform & Runestack Research
Email: nwalker85@gmail.com | ORCID: 0009-0005-0965-4424
Concept DOI: 10.5281/zenodo.23928301 | Version 2 DOI: 10.5281/zenodo.24044570
Publication Release Candidate — Phase B.3.2 — November 3, 2026

**Abstract**

Current paradigms in autonomous artificial intelligence rely predominantly on probabilistic software guardrails—natural language prompts, output schemas, and user-space virtual sandboxes—to constrain agent behavior. These mechanisms are vulnerable to direct prompt injection, semantic jailbreaks, and memory-state corruption. In this paper, we present **Odin OS**, a bare-metal, zero-allocation (`#![no_std]`) 64-bit RISC-V microkernel that anchors autonomous agent trust boundaries in processor Physical Memory Protection (PMP) comparators. Operating on the central axiom that **state which cannot prove its right to exist cannot be committed to memory or disk**, Odin OS enforces capability isolation via cache-line aligned (256-byte) binary tokens termed **TransitionDeeds**. We establish the primary falsifiable claim: **an untrusted U-Mode agent executing arbitrary instructions cannot commit a store to physical memory outside its explicitly authorized TransitionDeed window or after lease revocation without triggering an architectural CPU access fault (`mcause = 7`) that aborts the write and leaves target memory unmodified.**

We present seven core contributions: (1) **Cryptographic Authorization**: Integration of RFC 8032 Ed25519 digital signature verification in pure `#![no_std]` without heap allocation (`no_alloc`), bound to a 32-byte pinned root public key, with decoupled Policy Decision Point (PDP) authority isolation; (2) **Bi-Directional $W \oplus X$ Privilege Configuration**: Domain partitioning configuring user code as read-execute (`R-X`, store attempts fault with `mcause = 7`), while user stack memory is read-write non-executable (`RW-`, instruction fetch attempts fault with `mcause = 1` and are safely redirected to caller return addresses); (3) **Dynamic Capability Revocation**: Privileged syscall interface (`SYS_REVOKE_LEASE`, `ecall 13`) that dynamically collapses PMP comparator windows to `OFF`; (4) **17-Case Adversarial Matrix Evaluation**: Bare-metal QEMU verification demonstrating that all 17 adversarial test cases passed under emulation, comprising processor-enforced access faults and software-enforced authorization checks; (5) **Machine-Checked Formal Properties**: Z3 SMT models (`QF_BV`) verifying bounded integer overflow immunity, memory disjointness, and anti-replay sequence monotonicity; (6) **Mutation Testing Sensitivity**: Controlled injection of 15 architectural and cryptographic mutations, with 15 out of 15 detected and killed by the test suite; and (7) **End-to-End LLM Bridge**: Live coupling to local `google/gemma-4-e4b` inference, demonstrating that authorized memory stores execute under configured PMP permissions without per-store kernel mediation, while out-of-bounds mutation attempts are rejected at the PDP or trapped by processor PMP checks with no modification to inspected memory locations.

## 1 Introduction & Problem Statement

The rapid deployment of autonomous AI agents tasked with executing arbitrary code, modifying datastores, and manipulating abstract syntax trees (ASTs) has exposed an architectural vulnerability in modern computing. Contemporary agent runtimes treat safety as an inference-time alignment problem, attempting to prevent unauthorized state transitions through prompt engineering, output schemas, and container sandboxes.

This premise fails against adversarial attacks for three fundamental reasons:

1. **Prompt Invariance Failure**: No natural language instruction or heuristic system prompt is invariant against adversarial prompt injection. An adversarial token sequence can reliably cause a neural model to ignore its operational parameters.
2. **Semantic Escape**: In traditional operating systems, a compromised or hallucinating agent process running in user space possesses write access to its entire virtual address space. Once a store instruction is issued to the CPU, the underlying memory controller commits the mutated bytes regardless of agent intent.
3. **Post-Hoc Remediation**: Existing detection systems operate **after** memory has been mutated, relying on logging, rollbacks, or audit trails that incur latency and fail to prevent state corruption.

To resolve this vulnerability, we invert the trust model. Rather than attempting to guarantee that neural models will never produce malicious or errant store instructions, we construct an execution substrate where processor Physical Memory Protection restricts U-Mode accesses and raises architectural access faults when configured permissions are violated.

## 2 Threat Model & Trust Assumptions

### 2.1 The Central Axiom & Security Invariant

Odin OS is constructed around a single deterministic axiom:

> ***"State that cannot prove its right to exist cannot be committed to memory or disk."***

This enforces the fundamental security invariant:

$$\text{Agent Request} \neq \text{Authority Grant}$$

### 2.2 Trust Boundary Classification

Components are strictly categorized across five trust boundaries:

- **Policy Decision Point (PDP)**: **Trusted Authority**. Evaluates agent mutation requests against declarative security policies, manages monotonic sequence counters, and holds the root Ed25519 signing key. Operates in two modes: Mode A (Deterministic Research Fixture) using a documented test key for reproducible evaluation, and Mode B (Independently Provisioned Authority) with an external CSPRNG seed.
- **CPU Hardware (PMP Comparators / CSRs)**: **Trusted Enforcer**. Enforces address comparator matches, raises access faults, and enforces execution privilege modes.
- **M-Mode Microkernel (TCB)**: **Trusted Verifier & Enforcer**. Verifies Ed25519 signatures against the pinned root public key, programs PMP registers, and controls monotonic replay sequences.
- **Autonomous Agent (U-Mode Task)**: **Untrusted (Adversary)**. Executes untrusted model instructions and arbitrary arithmetic. Possesses zero signing keys and zero capability to modify PMP registers.
- **Host / Python Bridge**: **Untrusted Transport Channel**. Transports model tokens and capability requests; possesses zero private keys and is assumed to be completely compromiseable.

### 2.3 Enforcement Boundary: Spatial PEP vs. Semantic PDP

A critical architectural boundary governs Odin OS: the formal distinction between Policy Enforcement (PEP) and Policy Decision (PDP):

- **Spatial & Temporal PEP (Microkernel & PMP)**: Odin OS operates strictly as a hardware-enforced Policy Enforcement Point. It guarantees spatial memory confinement (stores cannot retire outside authorized byte spans) and temporal containment (stores cannot occur before lease entry or after revocation). Crucially, the PEP is agnostic to higher-level business or application semantics: it enforces physical capability constraints, not whether a proposed mutation makes contextual sense.
- **Semantic PDP (Host Authority)**: The Policy Decision Point evaluates declarative domain policies (e.g., agent identity, AST anchor legitimacy, allowed opcodes, lease duration caps) prior to minting and signing a TransitionDeed.
- **Application Semantic Correctness**: Establishing whether an authorized business action is contextually “appropriate” (e.g., high-level semantic truth, application invariants) is explicitly decoupled from the microkernel’s TCB. If an agent is tricked into requesting an unauthorized offset, the PDP denies the deed; if an agent attempts a store without or beyond an active deed, the CPU PMP traps the access.

## 3 TransitionDeed Protocol

State transitions in Odin OS are mediated through **TransitionDeeds**. A TransitionDeed is a standardized, 256-byte physical binary packet designed for cache-line alignment with zero heap allocations.

| Offset | Field | Type | Description |
|---|---|---|---|
| 0x00..0x02 | magic | [u8; 2] | Magic identifier 0x59 0x47 (“YG”) |
| 0x02..0x03 | version | u8 | Protocol specification version (0x02) |
| 0x03..0x04 | flags | u8 | Capability flags (MUTATION, AST_STRICT) |
| 0x04..0x08 | sequence | u32 | Monotonically increasing deed nonce |
| 0x08..0x18 | issuer_did | [u8; 16] | Cryptographic identity of the issuer |
| 0x18..0x28 | target_urn | [u8; 16] | Target entity uniform resource name |
| 0x28..0x48 | prev_deed_hash | [u8; 32] | Cryptographic hash of prior deed |
| 0x48..0x68 | base_digest | [u8; 32] | SHA-256 pre-mutation state digest |
| 0x68..0x6C | lease_cycles | u32 | Execution cycle lifetime bound |
| 0x6C..0x70 | operation | u8 + u8 + u16 | Opcode, escalation level, reserved |
| 0x70..0x74 | anchor_id | u32 | Target AST Anchor identifier |
| 0x74..0x78 | target_offset | u32 | Authorized buffer start offset |
| 0x78..0x7C | target_length | u32 | Authorized buffer span length |
| 0x7C..0x9C | new_digest | [u8; 32] | Expected post-mutation state digest |
| 0x9C..0xDC | signature | [u8; 64] | RFC 8032 Ed25519 digital signature |
| 0xDC..0x100 | padding | [u8; 36] | Zero padding to align to 256 bytes |

Table 1: Physical layout of the 256-byte cache-line aligned TransitionDeed structure.

## 4 Kernel Architecture & PMP Enforcement

### 4.1 Privilege Domain Layout

Odin OS partitions physical memory into five isolated privilege domains:

| Domain | Physical Span | Mode | PMP Mode | Privilege & Architectural Purpose |
|---|---|---|---|---|
| Domain 0 | 0x10000000 .. 0x10001000 | M-Mode | Default / --- | UART0 MMIO (Revoked from U-Mode) |
| Domain 1 | 0x80000000 .. 0x80010000 | M-Mode | Default / --- | Privileged Kernel: Code, Stack, Keys, Traps |
| Domain 2 | 0x80010000 .. 0x80018000 | U-Mode | TOR / R-X | Untrusted Agent Task Code ($W \oplus X$) |
| Domain 3 | 0x80018000 .. 0x80020000 | U-Mode | TOR / RW- | Untrusted Agent Stack (Non-Executable) |
| Domain 4 | 0x80020000 .. lease_start | None | TOR / --- | Gated Pre-Lease AST Boundary |
| Domain 5 | lease_start .. lease_end | U-Mode | TOR / RW- | Explicitly Authorized Transition-Deed Slice |
| Perimeter | All other physical addresses | None | OFF / --- | Processor Default Deny (No Access) |

Table 2: Hardened 7-Region PMP Layout and Privilege Partitioning.

### 4.2 Hardware W^X Stack Execution Prevention & Trap Handling

When user mode attempts to execute code from the user stack (`0x80018100`), the CPU instruction fetch unit triggers an Instruction Access Fault (`mcause = 1`). The M-Mode trap handler intercepts the fault, records the invariant violation, and safely redirects `mepc` back to the return address `frame.ra` in `.user_text`:

```
if mcause == 1 && mepc >= USER_STACK_START as u64 && mepc < USER_STACK_END as u64 {
    unsafe { core::arch::asm!("csrw mepc, {0}", in(reg) frame.ra); }
    return;
}
```

This guarantees that stack execution attempts cannot escalate privileges or enter infinite fault loops.

### 4.3 Temporal Lease Revocation (`SYS_REVOKE_LEASE`)

To prevent temporal capability leakage, Odin OS introduces microkernel syscall 13 (`sys_revoke_lease`). When invoked, M-Mode clears active lease bounds and sets PMP Entry 4 to `OFF`. Subsequent store attempts to the previously valid lease slice immediately trigger an architectural Store Access Fault (`mcause = 7`).

## 5 Cryptographic Authorization & Authority Isolation

### 5.1 Independent Policy Decision Point (PDP) Architecture

In compliance with the invariant $\text{Agent Request} \neq \text{Authority Grant}$, Odin OS decouples agent token generation from cryptographic signing. The private signing key is isolated within an independent Policy Decision Point service (`pdp_authority_service.py`). Neither the untrusted agent nor the communication bridge possesses access to private key material.

The PDP supports two operational modes:

1. **Mode A (Deterministic Research Fixture)**: Employs a public, documented test key (`b"odin_root_authority_seed_v2_key!"`) ensuring offline, deterministic test reproducibility.
2. **Mode B (Independently Provisioned Authority)**: Loads an external 32-byte CSPRNG seed stored outside the repository and injected into the microkernel at compile time via `build.rs` (`ODIN_ROOT_PUBKEY_HEX`). Mode B fails closed if key material is missing or invalid.

When an agent proposes a state mutation, the request is evaluated against a default-deny policy registry enforcing: identity verification, AST anchor whitelisting, spatial lease clamping, temporal caps, and monotonic sequence allocation. Only approved requests receive signatures. In empirical testing across 12 attack vectors (unauthorized anchors, invalid opcodes, oversized mutations, excessive lease durations, rogue agent identities, out-of-bounds offsets, zero-length mutations, post-sign tampering, sequence restarts, and Mode B key separation), the PDP rejected all 12 evaluated adversarial cases.

### 5.2 Canonical Domain Separation & Zero-Allocation Verifier

The signed message encompasses 172 contiguous bytes:

$$\text{Message} = \text{DOMAIN_SEPARATOR}_{16\text{B}} \,\|\, \text{Metadata}_{156\text{B}}$$

where `DOMAIN_SEPARATOR` is `"ODIN_DEED_V2\0\0\0\0"`. Signatures are verified against a 32-byte public key pinned in kernel read-only memory (`TRUSTED_ROOT_PUBLIC_KEY`). The verifier executes via `ed25519-compact` in pure `#![no_std]` with zero heap allocations.

## 6 Experimental Methodology

Our experimental evaluation executes on a 64-bit RISC-V platform (`riscv64gc-unknown-none-elf`) under QEMU 11.1.3 `virt`. Evidence is audited via a two-layer verification pipeline:

- **Layer A (In-Kernel)**: Direct M-Mode observation of trap causes, probed addresses, readback values, and memory preservation.
- **Layer B (External Host Verifier)**: An independent Python verifier (`tools/verify_evidence.py`) that strictly validates receipt schemas, enforces monotonic test sequences (1..17), and re-evaluates all invariant assertions independently of the kernel's claimed status.

### 6.1 Claim-to-Evidence Mapping

To provide peer reviewers with an explicit, bounded starting point for falsification, Table 3 maps each core architectural property to its trusted components, empirical test artifacts, and explicit non-scope boundaries:

| ID | Claim / Property | TCB & Assumptions | Supporting Artifact / Test | Explicitly Outside Scope |
|---|---|---|---|---|
| C1 | Spatial Confinement | RISC-V PMP comparators, M-Mode trap handler | Adversarial Tests 1–4, 9, 10; Z3 Proof 1 | Timing side-channels; non-atomic split stores on unverified silicon |
| C2 | Temporal Revocation | M-Mode syscall 13 (SYS_REVOKE_LEASE) | Adversarial Test 16; Mutation M09 | Autonomous timer-interrupt cycle preemption (mcycle) |
| C3 | Cryptographic Authority | ed25519-compact (no_std), pinned root key | Adversarial Tests 11, 13, 14; Mutations M01, M02, M11–M13 | Host signing key compromise outside the microkernel |
| C4 | Anti-Replay Nonce | In-kernel monotonic sequence counter last_seq | Adversarial Test 12; Mutation M03; Z3 Proof 2 | NVRAM counter rollback across reset without re-keying |
| C5 | Bi-directional $W \oplus X$ | PMP Entry 1 (R-X), Entry 2 (RW-) | Adversarial Tests 7 & 8; Mutations M06, M15 | Return-Oriented Programming (ROP) in valid user text |
| C6 | Semantic Appropriateness | External Policy Decision Point (PDP) | PDP Test Suite (12 attack vectors rejected) | **Outside Odin OS PEP scope.** Kernel enforces capability bounds, not business logic. |

Table 3: Claim-to-evidence mapping across TCB assumptions, empirical artifacts, and non-goals.

# 7 Adversarial Results

| # | Test Invariant Case | Target Addr | Width | Exp Fault | Obs Fault | mcause | Status |
|---|---|---|---|---|---|---|---|
| 1 | Exact Lower Bound | 0x80020040 | 1B | No | No | 0x00 | **PASS** |
| 2 | Exact Upper Bound - 1 | 0x8002005F | 1B | No | No | 0x00 | **PASS** |
| 3 | One Byte Below Lease | 0x8002003F | 1B | Fault | Fault | 0x07 | **PASS** |
| 4 | One Byte Above Lease | 0x80020060 | 1B | Fault | Fault | 0x07 | **PASS** |
| 5 | Kernel Code Write | 0x80000000 | 1B | Fault | Fault | 0x07 | **PASS** |
| 6 | Kernel Stack Write | 0x8000FFF8 | 1B | Fault | Fault | 0x07 | **PASS** |
| 7 | User Code Write ($W \oplus X$) | 0x80010000 | 1B | Fault | Fault | 0x07 | **PASS** |
| 8 | User Stack Execute ($W \oplus X$) | 0x80018100 | 0B | Fault | Fault | 0x01 | **PASS** |
| 9 | Unmapped Perimeter Write | 0x80030000 | 1B | Fault | Fault | 0x07 | **PASS** |
| 10 | Multi-byte Overlap Crossing | 0x8002005E | 4B | Fault | Fault | 0x07 | **PASS** |
| 11 | Forged Deed Issuer DID | 0x00000000 | 0B | Reject | Reject | Syscall / 1 | **PASS** |
| 12 | Replayed Deed Sequence | 0x00000000 | 0B | Reject | Reject | Syscall / 2 | **PASS** |
| 13 | Forged Deed Invalid Ed25519 | 0x00000000 | 0B | Reject | Reject | Syscall / 4 | **PASS** |
| 14 | Deed Tampered Metadata Ed25519 | 0x00000000 | 0B | Reject | Reject | Syscall / 4 | **PASS** |
| 15 | U-Mode CSR Access pmpcfg0 | 0x00000000 | 0B | Fault | Fault | 0x02 | **PASS** |
| 16 | Expired Lease Revocation | 0x80020040 | 1B | Fault | Fault | 0x07 | **PASS** |
| 17 | U-Mode UART0 MMIO Write | 0x10000000 | 1B | Fault | Fault | 0x07 | **PASS** |

Table 4: Empirical receipts from the 17-case adversarial verification matrix evaluated under QEMU (17/17 PASS).

## 7.1 Architectural Analysis of Multi-Byte Overlap Crossing (Test 10)

A 4-byte store at 0x8002005E straddles the lease boundary (0x80020060), with bytes [0x5E..0x5F] inside the lease and [0x60..0x61] outside. Because 0x8002005E is 2-byte aligned but not 4-byte aligned ($0x5E \neq 0 \bmod 4$), this instruction tests hardware behavior across an asymmetric protection boundary.

Under the **RISC-V Unprivileged Architecture Specification (Section 2.6)**:

*"Misaligned memory accesses are not guaranteed to be atomic."*

In our QEMU 11.1.3 virt evaluation, the processor performs address checking across split memory accesses prior to retiring the store, raising a Store Access Fault (mcause = 7) and leaving target memory unmodified (0x00). We verified that an unaligned 4-byte store entirely within the lease (0x80020052) succeeds without fault, demonstrating that QEMU supports unaligned accesses and that the fault at 0x8002005E is triggered solely by the PMP boundary violation.

**Physical Silicon Implication**: While software simulation in QEMU enforces all-or-nothing rollback on this split access, physical RISC-V microarchitectures are not architecturally bound by the base ISA to guarantee atomicity for misaligned stores. A physical core may either trap with `mcause = 6` (Store Address Misaligned) or commit the lower bytes before triggering a PMP fault on the upper half. Consequently, Odin OS establishes the architectural requirement that robust production deployments must couple PMP hardware fences with software-level staging buffers (shadow ASTs) to ensure strict transaction atomicity across arbitrary silicon implementations.

## 8 Mutation Testing

To evaluate test suite sensitivity, we injected 15 deliberate architectural and cryptographic faults:

| ID | Injected Architectural Mutation | Detecting Invariant Test | Result |
|---|---|---|---|
| M01 | Bypass issuer DID check | Test 11 (FORGED_DEED_UNKNOWN_ISSUER) | KILLED |
| M02 | Disable Ed25519 signature verification | Tests 13 & 14 (FORGED_DEED_ED25519) | KILLED |
| M03 | Permit sequence reuse (< instead of <=) | Test 12 (REPLAYED_DEED_SEQUENCE) | KILLED |
| M04 | Wrapping addition on bounds (disable checked) | prop_deed_overflow_prevention | KILLED |
| M05 | Expand PMP lease by 1 byte in setup_pmp | Test 4 (ONE_BYTE_ABOVE_HALT) | KILLED |
| M06 | Configure user code as RWX (0x0F) | Test 7 (USER_CODE_WRITE_HALT) | KILLED |
| M07 | Map UART MMIO in U-Mode | Test 17 (U_MODE_UART_MMIO_WRITE) | KILLED |
| M08 | Corrupt register restore in trap_entry | Architectural Register Invariants | KILLED |
| M09 | Skip PMP reprogramming on lease revocation | Test 16 (EXPIRED_LEASE_REVOCATION) | KILLED |
| M10 | Tamper with receipt status emission | Dynamic Invariant Assertions | KILLED |
| M11 | Mutate domain separator string | Host Test 8 (Cross-protocol) | KILLED |
| M12 | Truncate canonical payload slice | Host Test 20 (Valid signature) | KILLED |
| M13 | Corrupt pinned root public key | Matrix Test 1 & Host Test 20 | KILLED |
| M14 | Inject forged PASS into external Layer B trace | External Verifier (verify_evidence.py) | KILLED |
| M15 | Omit stack execution trap redirection | Matrix Test 8 (Infinite loop) | KILLED |

Table 5: Controlled mutation testing results demonstrating detection sensitivity across 15 injected faults (15/15 killed).

## 9 Formal Assurance

Using the Z3 Theorem Prover version 5.3.1 (`QF_BV`), we verified five machine-checked properties of the formal SMT model:

1. **Integer Overflow Bound**: Verified that under the model's assumptions, checked addition guarantees $\text{end_offset} > \text{target_offset}$ (`unsat`).
2. **Modeled Memory Disjointness**: Verified that for modeled address ranges, an accepted capability lease is disjoint from modeled privileged addresses (`unsat`).
3. **Modeled Buffer Containment**: Verified that modeled accepted leases remain within modeled buffer bounds (`unsat`).
4. **Identical Sequence Non-Replay**: Verified that resubmission of an identical sequence value is unsatisfiable under strict monotonicity (`unsat`).
5. **Stale Sequence Non-Replay**: Verified that any sequence number $s \leq \text{last_sequence}$ is unsatisfiable under strict monotonicity (`unsat`).

## 10 Limitations

1. **Simulation vs. Silicon**: All empirical measurements were conducted under QEMU 11.1.3 `virt`. Hardware characteristics such as non-zero grain size ($G \geq 4$), store pipeline write-buffer draining, and physical bus split-access behavior require physical silicon validation.
2. **Autonomous Timer Expiration**: Capability leases are revoked explicitly via microkernel syscalls (`sys_revoke_lease`); autonomous timer-interrupt cycle counting (`mcycle`) is designed for future work.
3. **Spatial vs. Semantic Integrity**: PMP enforces spatial write boundaries (`where` writes occur); verifying that RAM contents bitwise match cryptographic digests (`base_digest -> new_digest`) requires shadow AST staging buffers and commit barriers, specified as proposed architecture.

## 11 Related Work

- **seL4 Microkernel**: Formal verification of capability-based microkernel isolation. Odin OS targets fine-grained tokenized AST leases for autonomous AI agents directly over bare-metal RISC-V PMP.
- **CHERI**: Fat-pointer capability hardware extensions. Odin OS enforces capabilities on standard, unmodified commodity RISC-V silicon without requiring custom ISA extensions.
- **Memory Protection Keys (Intel MPK)**: Hardware memory domain toggling. Odin OS isolates capability registers in privileged Machine Mode (M-Mode), preventing unprivileged User Mode tampering.

## 12 Future Hardware Directions

Phase C of the Odin OS research roadmap targets physical deployment on physical silicon and reconfigurable logic platforms:

- **AMD Kria KV260 FPGA/MPSoC**: Dual-rail architecture integrating ARM Cortex-A53 application cores with programmable logic (PL) for hardware-accelerated AST graph traversal, direct AXI memory interlocks, and cryptographic digest hashing engines.
- **Physical RISC-V Silicon**: Deployment on 64-bit RISC-V application platforms (e.g., Microchip PolarFire SoC, StarFive JH7110), addressing store-buffer pipeline fence draining (`fence rw, rw`), non-volatile sequence persistence in secure EEPROM/NVRAM, and hardware timer interrupt cycle bounds (`mcycle`).

## 13 Conclusion

By combining zero-allocation Ed25519 signature verification, bi-directional $W \oplus X$ privilege configuration, temporal capability lease revocation, and evaluating the prototype against a 17-case QEMU adversarial matrix, 15 mutation tests, and machine-checked formal SMT models, Odin OS demonstrates an empirical architecture for constraining untrusted autonomous agent execution within hardware-enforced spatial boundaries.

## 14 Reproducibility & Artifact Availability

The complete Odin OS codebase, linker scripts, formal SMT specifications, and Layer B external verifiers are open-source and permanently archived on Zenodo.

- **Permanent Archive DOI**: `https://doi.org/10.5281/zenodo.23928302`
- **Zenodo Deposit**: `https://zenodo.org/records/23928302`
- **Source Repository**: `https://github.com/nwalker85/odinos`
- **Target Triple**: `riscv64gc-unknown-none-elf`
- **Licensing**: Apache-2.0 (Microkernel & Verification Tools); CC BY 4.0 (Manuscript)
- **External Verifier**: `python3 tools/verify_evidence.py`
- **Formal Proof Command**: `z3 formal/deed_bounds_verification.smt2 && z3 formal/sequence_anti_replay.smt2`